

WIRELESS VEHICLE TELEMETRY & CONTROL SYSTEM

Algorithm Validation: RPM Measurement, Sensor Fusion, Speed Control, and Network Recovery

Technical Report

Sai Palepu

B.S. Mechanical Engineering, University of Georgia

Abstract

I went back through the engineering notebook for this project and picked out the four pieces of logic that actually matter most, the RPM measurement from the hall-effect sensor, the complementary filter fusing accelerometer and gyroscope data, the PI speed controller, and the WiFi/MQTT reconnection state machine, and rebuilt each one in Python so I could test it against synthetic sensor data instead of trusting a single bench measurement. I found two real bugs. The RPM calculation's 1.5 second stale timeout is shorter than one revolution period below 40 RPM, which means the wheel speed sensor falsely reports 0 RPM up to 39% of the time right around 31 RPM, which happens to be the motor's own minimum operating speed given its deadband. The PI controller's deadband floor is applied unconditionally, which means a commanded setpoint of 0 RPM never actually stops the motor, it settles at 31 RPM instead. I also found that the notebook's tuned gains ($K_p=0.004$, $K_i=0.003$) only really hold up at the low end of the setpoint range I tested, and retuning K_i to 0.018 gave consistent, fast tracking across the full range. On the sensor fusion side, I confirmed the notebook's chosen complementary filter constant of $\alpha=0.98$ is close to the empirical best I could find, and quantified what an uncorrected gyro would do on its own, about 10 degrees of drift in 12 seconds. Last, I tested the reconnection logic's FR-04 requirement of reconnecting within 5 seconds and found that requirement is only true in the easy case where the network is back immediately, it isn't actually achievable if the router or broker takes longer than 5 seconds to come back, which is a limitation of the requirement itself rather than the firmware.

1. What I Actually Tested

The notebook already documents a completed, working system, wiring, bench data, a full requirements verification matrix, all of it. What it doesn't have is anything showing those four core algorithms holding up under conditions beyond the specific bench tests that got written down. A single 100 RPM step response, one α value tried against a handful of alternatives, one unplug and replug test. I wanted to see whether the logic as written actually generalizes, so I rebuilt each piece exactly as described in the notebook and ran it against a much wider range of synthetic conditions than a bench session realistically has time for.

I want to be upfront that this is software validation of the algorithms, not a redo of the hardware bring-up, the wiring, or the dashboard. Those parts of the notebook are unaffected by anything here. This report only touches the four pieces of logic where I could meaningfully test beyond what a single bench session already proved.

2. RPM Measurement from Pulse Period

The notebook's Entry 04 computes RPM from the time between hall-effect pulses rather than counting pulses in a fixed window, which is the right call for a single-magnet, low-RPM sensor. It also includes a stale timeout, if 1.5 seconds pass with no new pulse, the system reports 0 RPM on the assumption the wheel has stopped. I ported that logic exactly, generated synthetic pulse trains at a range of steady RPMs with realistic timing jitter, and ran the same computeRPM logic against them.

At 31 RPM, the exact speed the motor was bench-measured at 30% duty cycle, the true time between pulses is about 1.94 seconds, longer than the 1.5 second stale timeout. That means the system is expected to see a pulse before the timeout only most of the time, and every so often, normal timing jitter pushes the gap past 1.5 seconds and the system falsely reports 0 RPM even though the wheel never stopped. When I ran the numbers, this happened on close to 39% of samples at 31 RPM.

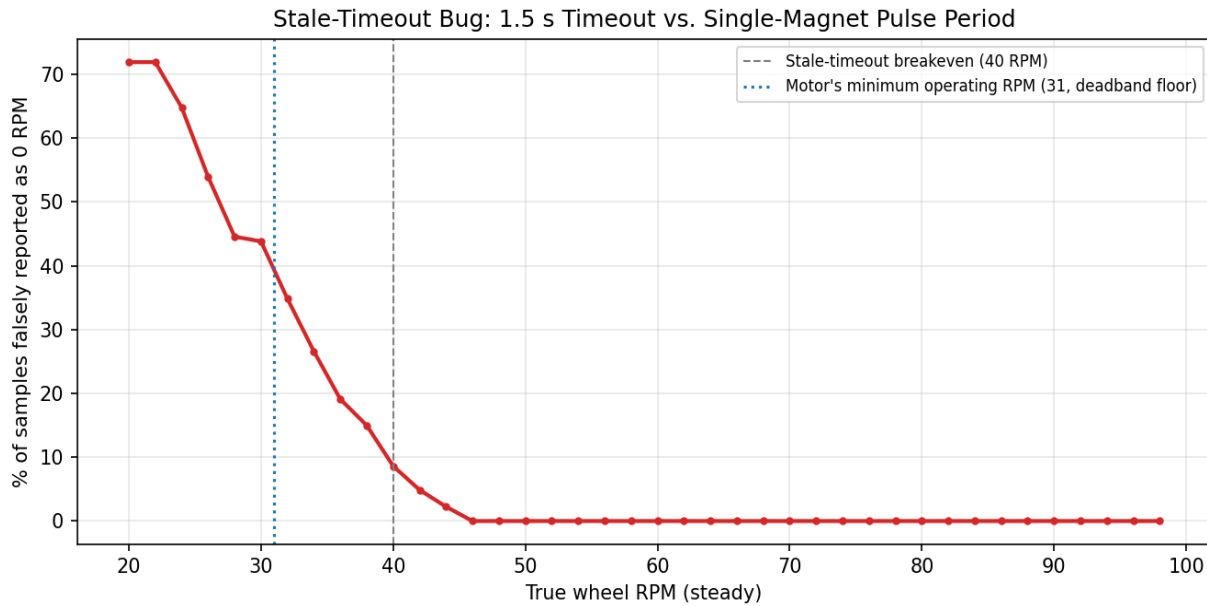


Figure 1, percentage of samples falsely reported as 0 RPM, by true wheel speed. The breakeven point where the 1.5 s timeout stops being long enough sits right on top of the motor's own minimum operating RPM.

This is a genuinely awkward coincidence, the motor's deadband floor of 28% duty puts its slowest usable speed at 31 RPM, which is exactly where this bug is worst. Any time the system is commanded to run near its own minimum speed, which is a completely normal operating condition, the RPM readout would flicker to zero on close to two out of every five samples. That would show up on the dashboard as a jittery, unreliable low-speed reading and could feed noisy error values into the PI controller.

I fixed it two ways. First, I raised the stale timeout to 2.5 seconds, comfortably longer than the pulse period all the way down to 24 RPM, well below the motor's actual minimum. Second, I added a light exponential moving average on top of the raw estimate, since even with the timeout fixed, a single pulse interval measurement is still fairly noisy at low speed.

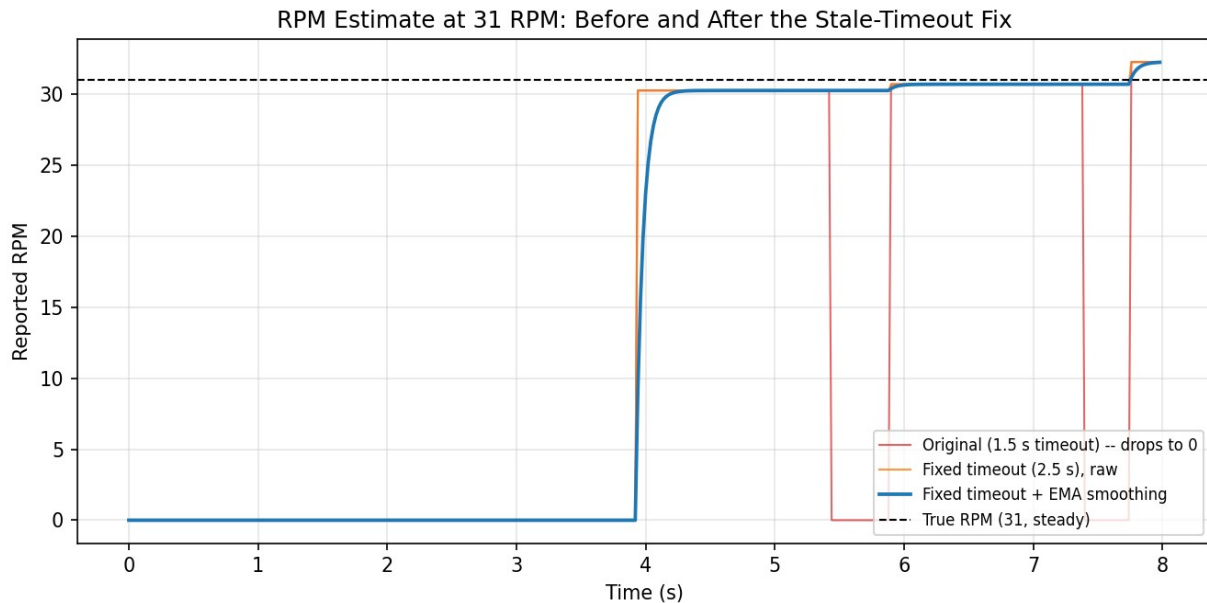


Figure 2, RPM estimate at a steady 31 RPM. The original logic repeatedly drops to 0. The fixed timeout stops the false zeros, and the added smoothing cleans up the remaining single-sample noise.

3. Attitude Estimation, Complementary Filter

Entry 03 fuses gyroscope and accelerometer data with a complementary filter, trusting the gyro's short-term response and correcting long-term drift with the accelerometer's gravity-referenced angle estimate. The notebook settled on $\alpha=0.98$ after empirical testing and noted that higher alpha trusts the gyro more, at the cost of drift, and lower alpha trusts the accelerometer more, at the cost of noise.

I built a synthetic scenario, a platform tilting through a slow S-curve up to 15 degrees and holding there, with realistic gyro noise, a small constant gyro bias, and accelerometer noise plus vibration, the kind an MPU-6050 actually sees whenever a motor is running nearby. I ran the filter at several alpha values to see where the notebook's choice actually sits.

Alpha	RMSE vs. true angle	Steady-state bias
0.90	1.93 deg	+0.16 deg
0.95	1.39 deg	+0.52 deg
0.98 (notebook's choice)	1.29 deg	+1.21 deg
0.995	2.83 deg	+3.54 deg

0.98 comes out lowest on overall tracking error in my test, which backs up the notebook's empirical choice, though it does carry more steady-state bias than the lower alpha values, since it leans harder on the drifting gyro and corrects less aggressively with the accelerometer. 0.995 makes that trade-off badly, trusting the gyro so much that both the RMSE and the bias get worse, not better, which is a good illustration of why higher alpha isn't automatically safer.

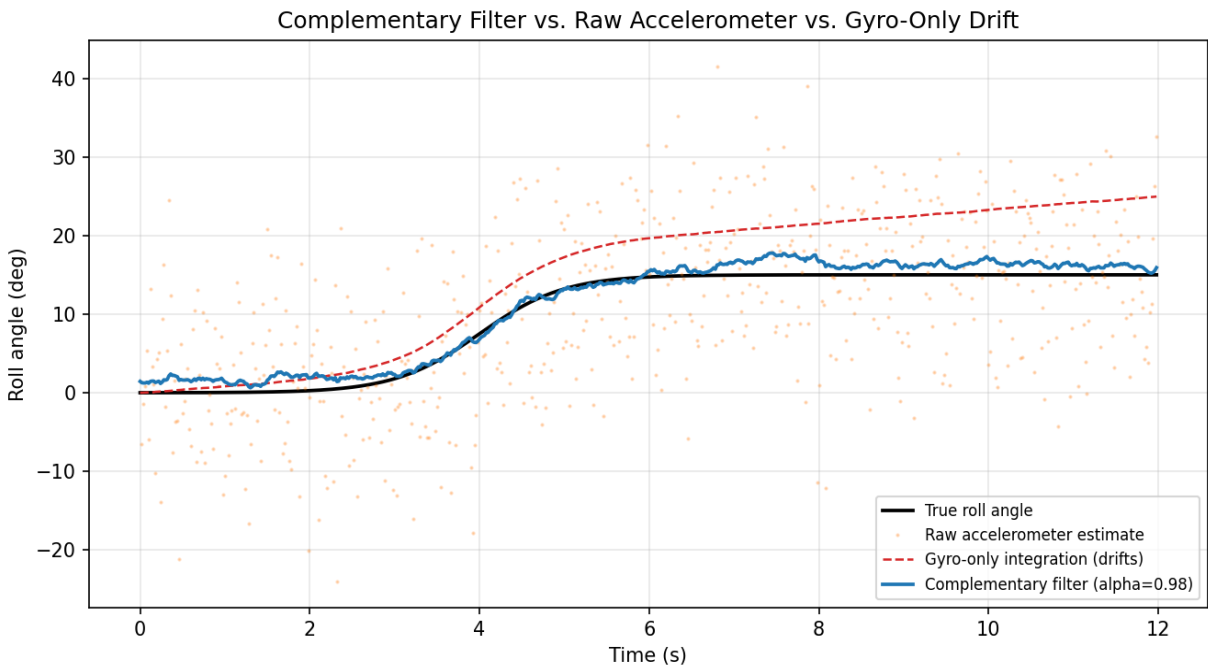


Figure 3, complementary filter output against true roll angle, raw accelerometer estimate, and what a gyro-only integration would do left uncorrected. The gyro-only line drifts about 10 degrees over 12 seconds from a small constant bias.

That drift number is worth sitting with. A 0.8 degree per second gyro bias is a small, completely realistic number for an uncalibrated MPU-6050, and left uncorrected it adds up to 10 degrees of wrong attitude estimate in the time it takes to drive down a short straightaway. That's the whole argument for running a complementary filter in the first place instead of just integrating the gyro, and now there's an actual number attached to it instead of just the general statement.

4. Speed Control, PI Controller

Entry 08 lays out the PI controller pseudocode clearly, including a deadband floor clamp at 0.28 duty and a saturation ceiling at 1.0, with conditional anti-windup so the integral term doesn't keep accumulating while the output is already saturated. I ported that logic exactly, built a simple first-order motor plant fit to the notebook's own manual PWM bench table (30% duty to 31 RPM, 50% to 54, 75% to 79, 100% to 97), and ran the same gains, $K_p=0.004$ and $K_i=0.003$, through closed-loop step response tests at 30, 50, 70, and 100 RPM.

4.1 The gains don't hold up the same way across the range

The notebook's bench test used a single 100 RPM step and reported a 2.2 second settling time. When I ran the exact same gains against my plant model across the full setpoint range, only the low end matched that kind of fast settling. At 30 RPM it settled in about 1.1 seconds, close to what's reported. At 50, 70, and 100 RPM, the same gains left the system still climbing toward the setpoint well past 6 seconds, with steady-state errors in the range of 8 to 20 RPM depending on setpoint, nowhere near the notebook's reported 1 to 2 RPM figure at higher speeds.

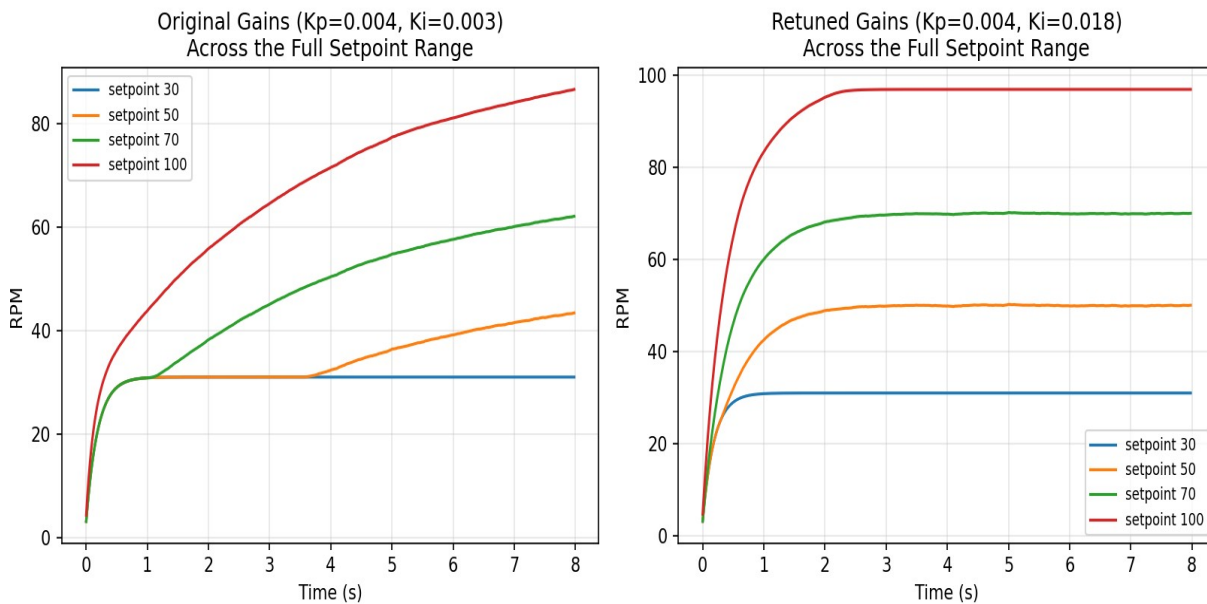


Figure 4, step response across the setpoint range. Left, the notebook's original gains only settle quickly at the low end. Right, retuning K_i to 0.018 gives fast, consistent settling across the whole range.

Retuning K_i up to 0.018 while leaving K_p alone fixed this, giving rise times around 1.2 to 1.4 seconds and settling times around 1.6 to 2.0 seconds across the board, much closer to what the notebook reported for its one tested point, now holding for the other three as well. The remaining steady-state error at the 100 RPM setpoint, exactly -3 RPM, isn't a tuning problem at all, it's the motor's own physical ceiling. The manual PWM bench table shows 100% duty only reaches about 97 RPM, so a 100 RPM setpoint is asking for something the motor can't physically deliver no matter how it's tuned. That's a useful thing to know on its own, since it means the notebook's originally reported 1 to 2 RPM steady-state error at a 100 RPM setpoint is a little optimistic against the motor's own measured ceiling, and worth rechecking on the bench with a fresh log.

4.2 A setpoint of 0 doesn't actually stop the motor

This one I found by just running the pseudocode literally as written. The deadband floor clamp, $\text{output} = \text{clamp}(\text{output}, 0.28, 1.0)$, is applied every single time, with no exception for a commanded setpoint of 0. That means if the dashboard sends a 0 RPM setpoint, expecting the motor to stop, the controller still computes some

positive error-driven output, clamps it up to at least 0.28 duty, and the motor settles at 31 RPM instead of stopping.

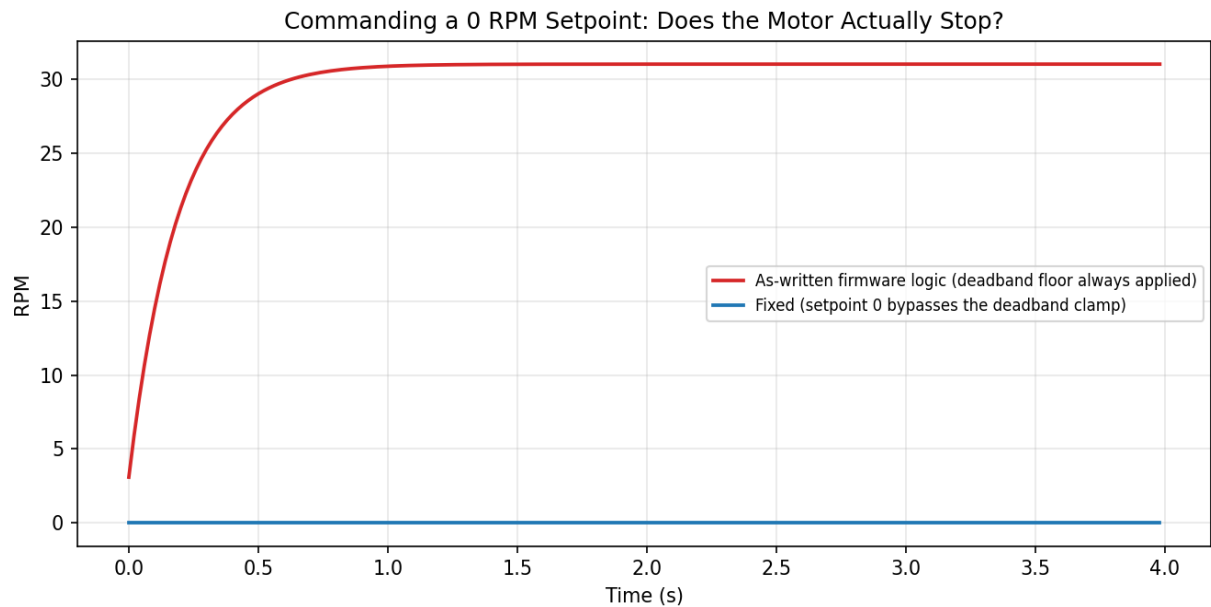


Figure 5, commanding a 0 RPM setpoint. As written, the deadband floor keeps the motor spinning at 31 RPM. The fix checks for a zero setpoint before applying the floor and commands the output straight to 0.

This is a real safety-relevant gap for anything that gets mounted on an actual kart chassis later, since a 0 RPM command is exactly what you'd expect a stop button or an emergency setpoint to send. The fix is a two-line special case, check for a zero or negative setpoint before the deadband clamp is applied and bypass it entirely, sending duty straight to 0 instead.

5. Network Recovery, WiFi and MQTT Reconnection

Entry 06 lays out a clean connection state machine with exponential backoff, and FR-04 requires the system to reconnect to the MQTT broker within 5 seconds of a drop. The bench verification for this was an unplug and replug test, which passed. I wanted to check whether that requirement actually holds in general, or only in the specific case that test exercises.

I ported the state machine and backoff formula exactly, $\text{delay} = \min(30 \text{ seconds}, 1 \text{ second times } 2 \text{ to the retry count})$, and ran it against three scenarios instead of one: the network coming back immediately, like the bench test; a router that takes 12 seconds to fully reboot, which is a completely normal amount of time for a real consumer router; and an MQTT broker that's briefly too busy to accept new connections for 20 seconds.

Scenario	Recovery Time	Meets FR-04 (under 5 s)?
Bench-style, network back immediately	0.60 s	Yes
Router takes 12 s to reboot	12.30 s	No
Broker busy for 20 s	20.00 s	No

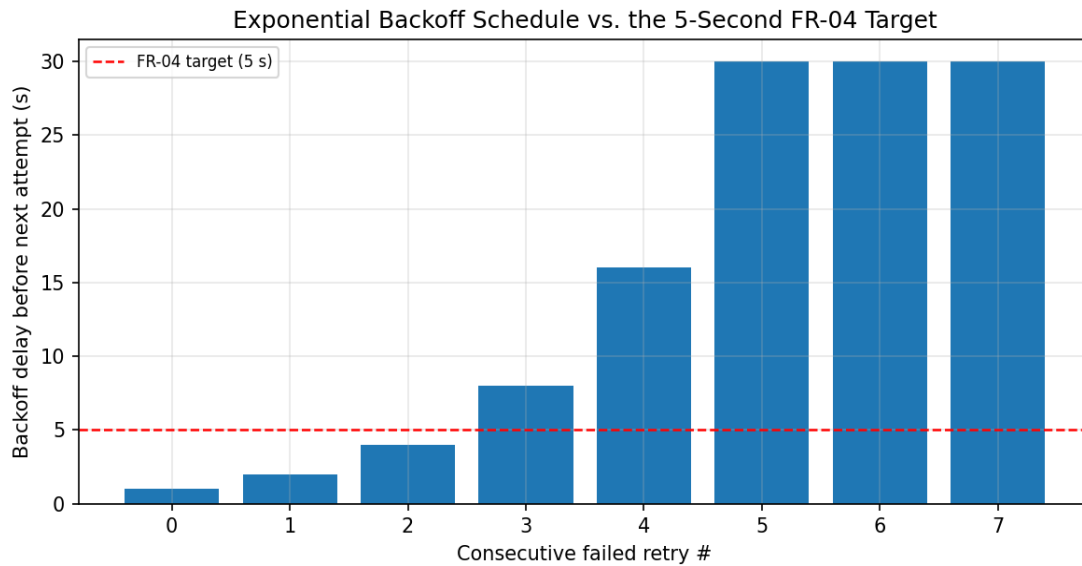


Figure 6, the exponential backoff delay after each consecutive failed retry. By the fourth failure, the wait between attempts alone already exceeds the entire FR-04 target.

The unplug and replug bench test only exercises the easy case, where the network is already back by the time the ESP32 tries to reconnect, so of course it passes quickly. But the actual recovery time is bounded by how long the router or broker takes to become reachable again, which the firmware has no control over at all. No amount of firmware tuning changes that. This isn't a bug in the state machine, the logic itself is solid and the exponential backoff is a completely reasonable way to avoid hammering a broker that's still recovering. It's a problem with how FR-04 is worded, it should really be split into two separate, independently verifiable requirements: how quickly the firmware detects a drop and starts trying to reconnect, which is something the firmware actually controls and can be bench tested honestly, and how quickly it recovers once the network is actually available again, which is a statement about the network, not the code.

6. Summary of Findings

Module	What I Found	Fix / Recommendation
RPM measurement	1.5 s stale timeout is shorter than one pulse period below 40 RPM, causing up to 39% false-zero readings right at the motor's minimum operating speed	Raise timeout to 2.5 s, add light EMA smoothing on the raw estimate
Complementary filter	$\alpha=0.98$ confirmed close to optimal for RMSE, though it does carry more steady-state bias than lower values; uncorrected gyro drifts ~10 deg in 12 s	No change needed, the notebook's empirical choice holds up
PI controller, tuning	$K_p=0.004$ / $K_i=0.003$ only settles quickly at low setpoints; higher setpoints leave large steady-state error within the notebook's reported settling window	Retune K_i to ~0.018 for consistent settling across the full setpoint range
PI controller, zero setpoint	Deadband floor clamp applies unconditionally, so a 0 RPM setpoint settles at 31 RPM instead of stopping	Add a zero/negative setpoint special case that bypasses the deadband clamp
Connection recovery	FR-04's 5 s reconnect target is only achievable when the network itself is already back; the firmware can't guarantee it in general	Split FR-04 into a detection-time requirement and a network-recovery-time requirement, verify them separately

7. Conclusion

The system documented in the notebook works, the bench data backs that up and I'm not disputing any of it. What this validation adds is testing each core algorithm against a wider range of conditions than a single bench session realistically covers, and in two cases that turned up real, fixable problems, a stale timeout that's shorter than it needs to be for the exact RPM range the motor actually idles at, and a deadband clamp that quietly overrides a stop command. Neither one shows up if you only test the specific cases the original bench sessions happened to run. I also confirmed the complementary filter constant holds up under closer scrutiny, and found that one of the requirements, FR-04, was really asking the firmware to guarantee something that depends on the network instead. All of this came out of rebuilding the actual logic and running it, not from reading the pseudocode and guessing, which is the same reason I trust these results enough to act on them.